EXPERIMENT NO. 7

MPI Collective Communication and Parallel Matrix Computations

HPC LAB

Name: Gaurav Madhav Patil
PRN: 23510040
Batch: B6

Solution / Performance Report

1. Aim

To understand and implement MPI collective communication operations and apply them to parallel matrix computations using distributed-memory programming.

Objectives

- Implement MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce().
- Implement parallel matrix-vector and matrix-matrix multiplication using MPI.
- Apply row-wise, column-wise, and block decomposition strategies.
- Implement non-uniform distribution using MPI_Scatterv().
- Measure execution time, speedup, and efficiency.
- Study communication overhead, load balance, scalability, matrix size, and process count effects.

2. Background / Use

In distributed-memory HPC systems, MPI processes have separate address spaces and therefore must explicitly exchange data. Collective operations provide standard mechanisms for distributing, broadcasting, gathering, and reducing data. Matrix computations are well suited to parallel execution because independent rows, columns, or blocks can be processed concurrently.

The basic HPC pattern used in this experiment is: Distribute → Compute → Communicate → Combine.

3. Preliminary Questions

1. What is collective communication in MPI?

Collective communication is communication involving a group of processes in an MPI communicator, such as broadcast, scatter, gather, and reduction.

2. How is collective communication different from point-to-point communication?

Point-to-point communication normally involves a sender and a receiver, while collective communication involves all processes in a communicator according to the selected collective operation.

3. What is the purpose of MPI_Bcast()?

MPI_Bcast() sends the same data from one root process to every process in the communicator.

4. Difference between MPI_Scatter() and MPI_Gather().

MPI_Scatter() distributes portions of data from the root to processes. MPI_Gather() collects corresponding data from processes at the root.

5. What does MPI_Reduce() do?

MPI_Reduce() combines values from all processes using an operation such as MPI_SUM, MPI_MAX, or MPI_MIN and returns the result to the root.

6. Why is MPI_Barrier() used?

MPI_Barrier() synchronizes processes by making each process wait until all processes in the communicator reach the barrier.

7. What is data decomposition?

Data decomposition divides a large data set into smaller portions so different processes can operate on different portions concurrently.

8. Why is matrix computation suitable for parallel processing?

Many matrix operations contain independent rows, columns, or blocks, allowing the work to be divided among processes.

9. What is row-wise decomposition?

Rows of a matrix are divided among processes, with each process computing the output associated with its assigned rows.

10. What happens if rows are not evenly divisible by processes?

Equal-size MPI_Scatter() distribution cannot directly handle the remainder. MPI_Scatterv() can assign different numbers of rows to different processes.

4. Section A — Task 1: Broadcast Using MPI_Bcast()

Process 0 initializes an integer value (100) and broadcasts it to all processes using MPI_Bcast(). The experiment was executed with 2, 4, and 8 processes.

Execution Results

Processes	Process Rank	Received Value
2	0	100
2	1	100
4	0	100
4	1	100
4	2	100
4	3	100
8	0	100
8	1	100
8	2	100
8	3	100
8	4	100
8	5	100
8	6	100
8	7	100

Observation: every process received the same value 100. The order of terminal output can vary because processes execute concurrently.

5. Section A — Task 2: MPI_Scatter() and MPI_Gather()

Process 0 creates the array [1, 2, ..., 16]. Four processes receive four elements each using MPI_Scatter(). Each process computes its local sum and the local results are collected using MPI_Gather().

Process	Received Data	Local Sum
P0	1 2 3 4	10
P1	5 6 7 8	26
P2	9 10 11 12	42
P3	13 14 15 16	58
Final	—	136

Verified final result: 10 + 26 + 42 + 58 = 136.

6. Section A — Task 3: Global Reduction Using MPI_Reduce()

The same distributed array was processed using MPI_Reduce() with MPI_SUM, MPI_MAX, and MPI_MIN.

Process	Local Sum	Local Maximum	Local Minimum
P0	10	4	1
P1	26	8	5
P2	42	12	9
P3	58	16	13

Reduction	Global Result
MPI_SUM	136
MPI_MAX	16
MPI_MIN	1

MPI_Gather() is useful when the individual process results are required at the root. MPI_Reduce() is preferable when only a final aggregate value such as sum, maximum, or minimum is required.

7. Section B — Task 4: MPI Matrix–Vector Multiplication

The operation is Y = AX. A is an N×N matrix and X and Y are vectors of length N. The MPI implementation uses row-wise decomposition: X is broadcast, rows of A are scattered, each process computes its local part of Y, and the results are gathered.

Sequential Execution Results

Matrix Size	Sequential Time (s)	Verification
500 × 500	0.002405	Y[0] = 25250, Y[499] = 25250
1000 × 1000	0.002982	Y[0] = 50500, Y[999] = 50500
2000 × 2000	0.007565	Y[0] = 101000, Y[1999] = 101000

MPI Execution Results

Matrix Size	Processes	MPI Time (s)	Verification
500 × 500	2	0.000918	25250
500 × 500	4	0.000586	25250
500 × 500	8	N/A — 500 not divisible by 8	—
1000 × 1000	2	0.003734	50500
1000 × 1000	4	0.003585	50500
1000 × 1000	8	0.001872	50500
2000 × 2000	2	0.014321	101000
2000 × 2000	4	0.014297	101000
2000 × 2000	8	0.015689	101000

Performance Table

Matrix Size	Processes	Sequential (s)	MPI (s)	Speedup	Efficiency
500	2	0.002405	0.000918	2.6198×	130.99%
500	4	0.002405	0.000586	4.1041×	102.60%
500	8	0.002405	N/A	N/A	N/A
1000	2	0.002982	0.003734	0.7986×	39.93%
1000	4	0.002982	0.003585	0.8318×	20.79%
1000	8	0.002982	0.001872	1.5929×	19.91%
2000	2	0.007565	0.014321	0.5282×	26.41%
2000	4	0.007565	0.014297	0.5291×	13.23%
2000	8	0.007565	0.015689	0.4822×	6.03%

Note: The measured efficiency above 100% for the 500×500 cases is retained because it is the value obtained from the recorded timings. The sequential implementation timed the multiplication using clock(), whereas the MPI timing included MPI communication as well; therefore the timing scopes are not perfectly identical.

8. Section B — Task 5: MPI Matrix–Matrix Multiplication

The operation is C = A×B. The MPI implementation uses row-wise decomposition: rows of A are scattered, B is broadcast to all processes, each process computes its assigned rows of C, and the rows of C are gathered.

Sequential Results

Matrix Size	Sequential Time (s)	Verification
500 × 500	0.227674	C[0][0] = C[499][499] = 1691750
1000 × 1000	1.821027	C[0][0] = C[999][999] = 3383500
2000 × 2000	13.781777	C[0][0] = C[1999][1999] = 6767000

MPI Results

Matrix Size	Processes	MPI Time (s)	Verification
500 × 500	2	0.115277	1691750
500 × 500	4	0.063077	1691750
500 × 500	8	N/A — 500 not divisible by 8	—
1000 × 1000	2	0.932707	3383500
1000 × 1000	4	0.566769	3383500
1000 × 1000	8	0.387884	3383500
2000 × 2000	2	7.344383	6767000
2000 × 2000	4	4.470197	6767000
2000 × 2000	8	3.761871	6767000

Performance Table

Matrix Size	Processes	Sequential (s)	MPI (s)	Speedup	Efficiency
500	2	0.227674	0.115277	1.9741×	98.70%
500	4	0.227674	0.063077	3.6094×	90.24%
500	8	0.227674	N/A	N/A	N/A
1000	2	1.821027	0.932707	1.9525×	97.63%
1000	4	1.821027	0.566769	3.2132×	80.33%
1000	8	1.821027	0.387884	4.6949×	58.69%
2000	2	13.781777	7.344383	1.8766×	93.83%
2000	4	13.781777	4.470197	3.0826×	77.06%
2000	8	13.781777	3.761871	3.6635×	45.79%

Observation: Increasing the number of MPI processes reduced matrix-matrix execution time for the tested sizes. The improvement is not perfectly linear because communication, synchronization, and other system overheads increase the cost of using more processes.

9. Section C — Task 6: Compare Different Matrix Decomposition Strategies

Three decomposition strategies were investigated on a 1000×1000 matrix using 4 MPI processes: row-wise, column-wise, and block-wise (2D).

Strategy	Matrix Size	Processes	Execution Time (s)	Sample Result
Row-wise	1000 × 1000	4	0.566769	3383500.00
Column-wise	1000 × 1000	4	0.504285	3383500.00
Block-wise (2D)	1000 × 1000	4	0.517364	3383500.00

All three implementations produced the same verified sample result. The column-wise implementation had the lowest measured time in this particular run, followed closely by block-wise and then row-wise.

Strategy	Advantages	Limitations / Considerations	Typical Use
Row-wise	Simple; natural for row-major storage; straightforward MPI_Scatter/Gather.	Requires suitable row distribution; communication includes distributing rows and collecting results.	Matrix-vector and row-partitioned matrix multiplication.
Column-wise	Can distribute output columns among processes.	Each process needs the complete A matrix; B columns are distributed; memory and communication requirements can be higher.	Column-oriented computations.
Block-wise (2D)	Divides work in two dimensions; good locality potential; scalable structure.	More complex indexing, communicator/grid management, and data movement.	Large-scale distributed matrix computations.

Task 6 Analysis

- Communication: row-wise and column-wise approaches require distribution of matrix data and collective communication. Block decomposition uses a process grid and communicates blocks.
- Computation: all strategies perform the same mathematical C = A×B operation, but the amount of local work and data required by each process differs.
- Memory: column-wise computation in this implementation keeps the complete A matrix available to each process; block decomposition stores smaller local blocks.
- Load balancing: with four processes and a 1000×1000 matrix, the tested decompositions were evenly partitioned.
- Execution time: the measured times were 0.566769 s row-wise, 0.504285 s column-wise, and 0.517364 s block-wise.
- Scalability: decomposition should expose enough independent work while keeping communication and synchronization manageable.

10. Section C — Task 7: Non-Uniform Distribution Using MPI_Scatterv()

For N = 10 and 3 processes, ten matrix rows cannot be divided equally. MPI_Scatterv() was used to distribute 4 rows to P0, 3 rows to P1, and 3 rows to P2. MPI_Gatherv() was then used to reconstruct the result vector.

Process	Rows Distributed	Rows Computed
P0	4	4
P1	3	3
P2	3	3

Result vector Y:

55.00 55.00 55.00 55.00 55.00 55.00 55.00 55.00 55.00 55.00

Verification: Y[0] = 55.00, Y[9] = 55.00

MPI_Scatter vs MPI_Scatterv

Aspect	MPI_Scatter	MPI_Scatterv
Data distribution	Equal amount to every process.	Different amounts can be sent to different processes.
Flexibility	Suitable when data divides evenly.	Suitable when data does not divide evenly.
Programming complexity	Simpler counts/displacements.	Requires send counts and displacements.
Real HPC applicability	Useful for regular, balanced partitions.	Useful for irregular data sizes, remainders, and uneven workloads.

11. Performance Analysis

The PDF requires execution time, speedup, efficiency, communication overhead, computation time, load imbalance, scalability, and the effects of matrix size and process count to be investigated.

Communication Overhead

MPI programs perform communication through collective operations such as broadcast, scatter, gather, reduction, and Scatterv/Gatherv. For relatively small computations, this communication can be significant compared with the local arithmetic work. This helps explain why adding processes does not always reduce the measured time.

Computation Time

Matrix-matrix multiplication performs substantially more arithmetic than matrix-vector multiplication. The measured sequential time for 2000×2000 matrix multiplication was 13.781777 s, compared with 0.007565 s for matrix-vector multiplication.

Load Imbalance

Equal row-wise partitioning provides balanced work when the matrix size is divisible by the process count. When it is not divisible, MPI_Scatterv() can distribute different row counts, as demonstrated by the 4/3/3 split in Task 7.

Scalability

The matrix-matrix experiment showed decreasing execution time as processes increased from 2 to 8 for the tested sizes. However, speedup was sublinear, showing that communication and other overheads prevent ideal linear scaling.

Effect of Matrix Size

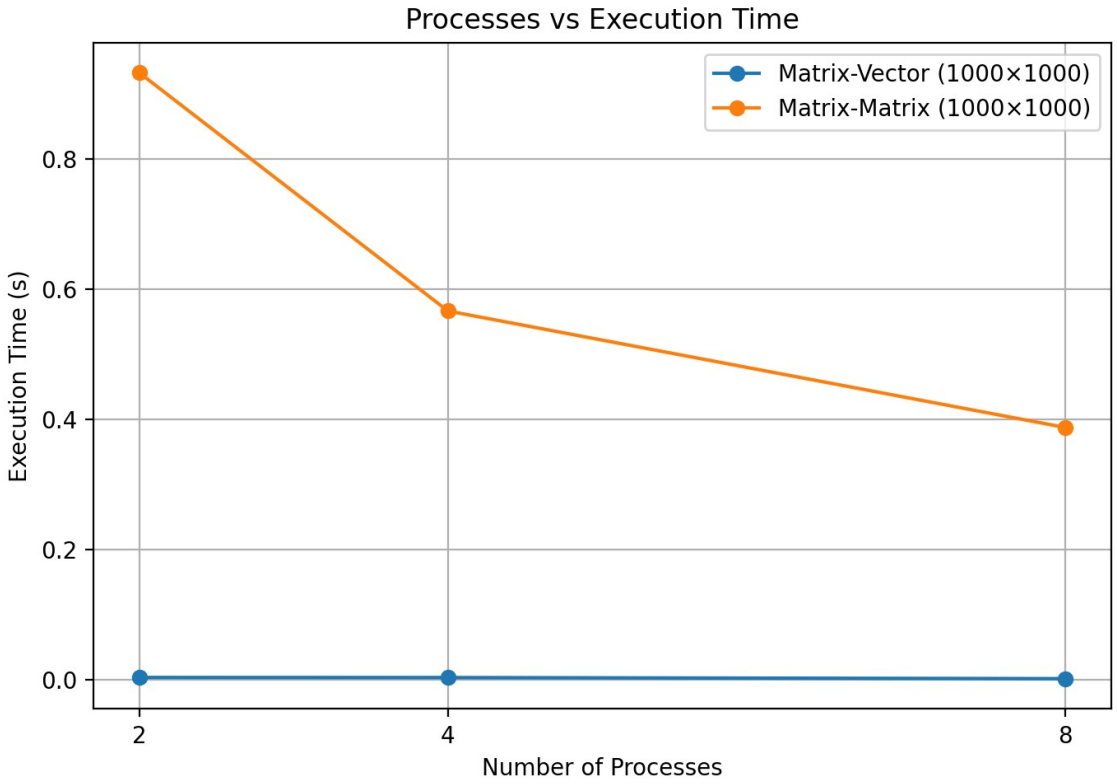
Larger matrices provide more computation that can be distributed. For sequential matrix multiplication, execution time increased from 0.227674 s (500×500) to 1.821027 s (1000×1000) and 13.781777 s (2000×2000).

Effect of Number of Processes

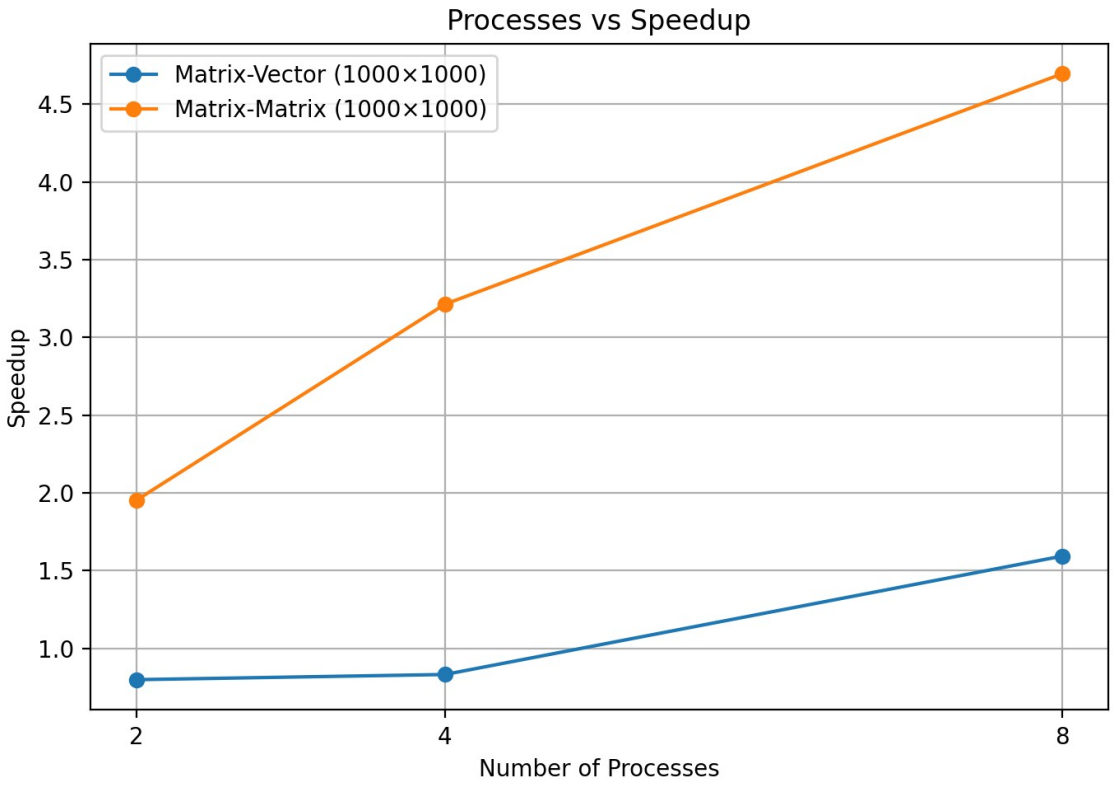
More processes can reduce computation per process, but the benefit depends on the problem size and communication cost. The 1000×1000 matrix-matrix case decreased from 0.932707 s with 2 processes to 0.566769 s with 4 and 0.387884 s with 8.

Required Graphs

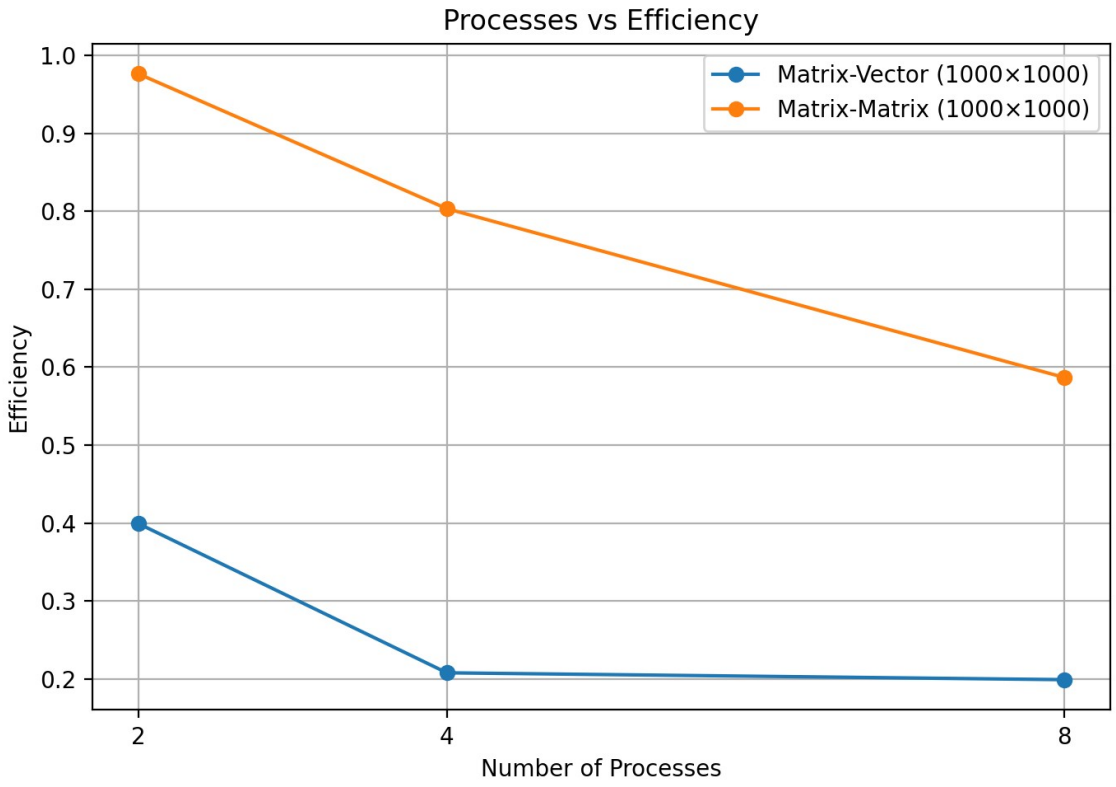
Processes vs Execution Time


Processes vs Execution Time
Matrix-Vector (1000×1000)
Matrix-Matrix (1000×1000)
Execution Time (s)
0.0
0.2
0.4
0.6
0.8
2
4
8
Number of Processes

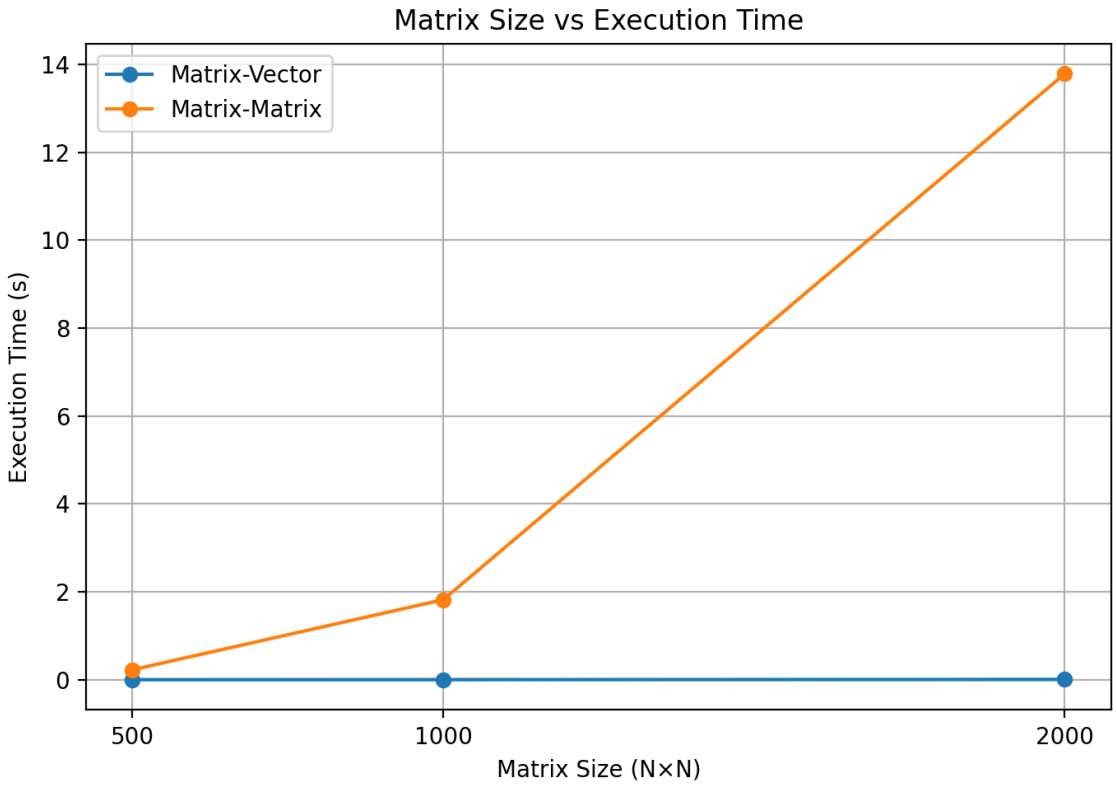
Processes vs Speedup



Processes vs Efficiency



Matrix Size vs Execution Time



12. AI/HPC Case Study — Deep Learning

Deep learning is a real-world application where distributed matrix computation and collective communication are heavily used. Training a neural network involves repeated matrix and vector operations over large numbers of training samples. The problem becomes computationally intensive because every training iteration may require forward propagation, loss calculation, backpropagation, and parameter updates over large tensors.

A common distribution strategy is data parallelism. The training dataset is divided into mini-batches and different processing units work on different portions of the data. Each worker can maintain a copy of the model and process its assigned mini-batch. For a distributed-memory system, MPI can coordinate communication between workers. Collective operations can distribute common data, gather results, or combine values such as gradients. For example, gradient information computed independently by multiple processes can be combined before model parameters are updated.

A suitable decomposition strategy is therefore data decomposition, with additional block-style decomposition inside large matrix operations. In a heterogeneous HPC system, GPUs can perform the intensive matrix calculations while MPI coordinates communication between machines or GPU-equipped nodes. This combination can allow a larger global batch of training data to be processed in parallel and can reduce the time required for training large models.

Parallel execution provides higher aggregate computational throughput and allows large datasets and models to be handled across multiple machines. However, scalability is limited by communication and synchronization. If workers frequently wait for gradient exchange or if network communication becomes a significant fraction of each iteration, adding more processes may produce smaller performance improvements. Load imbalance can also occur when different workers have unequal amounts of work. Therefore, practical distributed AI systems must balance computation, memory capacity, communication volume, synchronization frequency, and network bandwidth.

13. Discussion Questions

1. What is collective communication?

It is communication performed by a group of MPI processes in a communicator using operations such as broadcast, scatter, gather, or reduction.

2. How does MPI_Bcast() work?

The root process supplies the data and MPI_Bcast() makes that data available to every process in the communicator.

3. Differentiate MPI_Scatter() and MPI_Gather().

Scatter distributes different portions of root data to processes; Gather collects corresponding portions from processes at the root.

4. Differentiate MPI_Gather() and MPI_Reduce().

Gather collects individual values or arrays at the root, while Reduce combines values using an operation such as sum, maximum, or minimum.

5. Why is MPI_Bcast() useful in matrix multiplication?

When every process needs the same matrix, broadcasting avoids separately sending the same data to each process with individual point-to-point operations.

6. Why is row-wise decomposition suitable for matrix-vector multiplication?

Each output element depends on one row of A and the complete vector X, so rows can be distributed independently while X is broadcast to all processes.

7. What are the limitations of row-wise decomposition?

It requires appropriate row distribution, and communication plus synchronization overhead can limit scalability. It also becomes less convenient when rows cannot be divided evenly.

8. What is block decomposition?

Block decomposition divides a matrix into smaller two-dimensional submatrices and assigns blocks to processes.

9. Why can block decomposition improve data locality?

A process works repeatedly on nearby elements in smaller blocks, which can improve reuse of data in cache and reduce unnecessary memory accesses.

10. What is the purpose of MPI_Scatterv()?

MPI_Scatterv() distributes different numbers of elements to different processes using per-process counts and displacements.

11. What happens when the number of data elements is not evenly divisible by the number of processes?

An equal Scatter partition cannot distribute the remainder equally. Scatterv can assign different amounts so every element is handled.

12. Why does execution time sometimes stop decreasing as the number of processes increases?

Communication, synchronization, memory contention, and fixed overheads become more significant as process count increases.

13. How does communication overhead affect MPI scalability?

Communication consumes time that could otherwise be spent computing. As the process count increases, communication can limit speedup and efficiency.

14. How does MPI differ from OpenMP in terms of memory architecture?

MPI processes normally have separate memory spaces and communicate explicitly, whereas OpenMP threads normally share the memory of one process.

15. For a very large matrix, what factors should be considered when selecting a decomposition strategy?

Consider matrix size, available memory, communication volume, load balance, cache/data locality, process topology, scalability, and the cost of synchronization.

14. Reflection

1. What was the most important concept you learned from this assignment?

The most important concept was how MPI collective communication enables processes to distribute, share, and combine data in a distributed-memory program.

2. Which MPI collective operation did you find most useful and why?

MPI_Bcast() was particularly useful because common data such as a matrix or vector can be made available to all processes with one collective operation.

3. How is MPI matrix multiplication different from OpenMP matrix multiplication?

MPI uses separate processes with explicit data distribution and communication, while OpenMP normally uses threads sharing the same memory.

4. What happens when more MPI processes are added?

The computational work per process can decrease, but communication and synchronization overhead also increase. Therefore, execution time may improve by a diminishing amount.

5. Did increasing the number of processes always improve performance? Explain using your observations.

No. In the matrix-vector experiment, increasing the process count did not always improve the measured time. For example, the 2000×2000 case measured 0.014321 s with 2 processes, 0.014297 s with 4, and 0.015689 s with 8.

6. Which decomposition strategy would you select for a large matrix and why?

A block or 2D decomposition is a strong option for a large distributed matrix because it provides work in two dimensions and can improve locality, although the communication and implementation are more complex.

7. What communication bottlenecks did you observe?

The main communication operations were broadcasting common matrices/vectors and scattering/gathering matrix rows or blocks. These operations contribute overhead, especially when the computation is relatively small.

8. How could the implementation be further optimized for an HPC cluster?

Possible improvements include optimized block algorithms, nonblocking communication, better process-grid mapping, optimized memory access, tuned MPI collectives, larger problem sizes, and hybrid MPI+OpenMP or MPI+GPU execution.

15. Conclusion

The experiment demonstrated MPI collective communication and its application to distributed matrix computations. MPI_Bcast(), MPI_Scatter(), MPI_Gather(), and MPI_Reduce() were successfully implemented and verified. Matrix-vector and matrix-matrix multiplication were executed using row-wise MPI decomposition, while row-wise, column-wise, and block-wise decomposition strategies were compared for matrix multiplication. MPI_Scatterv() was successfully used to handle a non-uniform 4/3/3 row distribution.

The measured results show that increasing the process count can reduce matrix-matrix execution time, but speedup is not perfectly linear because of communication and synchronization overhead. Matrix-vector multiplication showed that for smaller or communication-dominated cases, adding processes may provide limited or inconsistent benefit. The decomposition experiment also showed that different data layouts can produce different measured execution times.

Overall, the experiment provided practical understanding of distributed-memory programming, collective communication, data decomposition, and MPI performance analysis, and demonstrated their relevance to large-scale HPC and AI workloads.